



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №5
з дисципліни
“Технології розроблення програмного забезпечення”
на тему “Редактор зображень”

Виконав
студент групи ІА–22:
Гордун Д.В.

Перевірив
Мягкий М.Ю.

Київ 2024

Хід роботи

Додав код за [посиланням](#)

«Анти-шаблони» проектування

Анти-патерни (anti-patterns), також відомі як пастки (pitfalls) - це класи найбільш часто впроваджуваних поганих рішень проблем. Вони вивчаються, як категорія, в разі коли їх хочуть уникнути в майбутньому, і деякі їхні окремі випадки можуть бути розпізнані при вивченні непрацюючих систем. Термін походить з інформатики, від авторів «Банди чотирьох» книги «Шаблони проектування», яка заклала приклади практики хорошого програмування. Автори назвали ці хороші методи «шаблонами проектування», і протилежними їм є «анти-патерни». Частиною хоршої практики програмування є уникнення анти-патернів.

Анти-патерни в управлінні розробки ПЗ:

- Дим і дзеркала (Smoke and mirrors): Демонстрація того, як будуть виглядати ненаписані функції (назва походить від двох улюблених способів, якими фокусники приховують свої секрети).
- Роздування ПЗ (Software bloat): Дозвіл наступним версіям системи вимагати все більше і більше ресурсів;
- Функції для галочки: Перетворення програми в конгломерат погано реалізованих і не пов'язаних між собою функцій (як правило, для того, щоб заявити в рекламі, що функція є);

Анти-патерни в розробці ПЗ:

- Інверсія абстракції (Abstraction inversion): Приховування частини функціоналу від зовнішнього використання, з надією на те, що ніхто не буде його використовувати;
- Невизначена точка зору (Ambiguous viewpoint): Подання моделі без специфікації її точки розгляду;
- Великий клубок бруду (Big ball of mud): Система зі структурою, яку не можна розпізнати;
- Блоб (Blob): див. Божественний об'єкт (God object);
- Бензинова фабрика (Gas factory): Необов'язкова складність дизайну;

- Затичка на введення даних (Input kludge): Забудькуватість в специфікації і виконанні підтримки можливого неправильного введення;

Анти-патерни в об'єктно-орієнтованому програмуванні:

- Базовий клас-утиліта (BaseBean): Спадкування функціональності з класу-утиліти замість делегування до нього;
- Виклик предка (CallSuper): Для реалізації прикладної функціональності методу класунащадка потрібно в обов'язковому порядку викликати ті ж методи класу-предка;
- Помилка порожнього підкласу (Empty subclass failure): Створення класу (в Perl), який не проходить «перевірку порожнечі підкласу» («Empty Subclass Test») через різницю у поведінці в порівнянні з класом, який успадковується від нього без змін;
- Божественний об'єкт (God object): Концентрація занадто великої кількості функцій в одній частині системи (класі);
- Об'єктна клоака (Object cesspool): перевикористання об'єктів, що знаходяться в непридатному для перевикористання стані;

Анти-патерни в програмуванні:

- Непотрібна складність (Accidental complexity): Внесення непотрібної складності в рішення;
- Дія на відстані (Action at a distance): Несподівана взаємодія між широко розділеними частинами системи;
- Накопичити і запустити (Accumulate and fire): Установка параметрів підпрограм в наборі глобальних змінних;
- Слепа віра (Blind faith): Недостатня перевірка (а) коректності виправлення помилки або (b) результату роботи підпрограми;
- Активне очікування (Busy spin): Споживання ресурсів ЦПУ (процесорного часу) під час очікування події, зазвичай за допомогою постійно повторюваної перевірки, замість того, щоб використовувати асинхронне програмування (наприклад, систему повідомлень або подій);
- Кешування помилки (Caching failure): Забувати скинути прапор помилки після її обробки;

Методологічні анти-патерни:

- Програмування методом копіювання-вставки (Copy and paste programming): Копіювання (і легка модифікація) існуючого коду замість створення спільних рішень;
- Дефакторінг (De-Factoring): Процес знищення функціональності і заміни її документацією;
- Золотий молоток (Golden hammer): Сильна впевненість в тому, що улюблене рішення універсально застосовується. Назва походить від приказки «коли в руках молоток, всі проблеми здаються цвяхами»;
- Фактор неймовірності (Improbability factor): Припущення про неможливість того, що спрацює відома помилка;
- Передчасна оптимізація (Premature optimization): Оптимізація на основі недостатньої інформації;

Анти-патерни управління конфігурацією:

- DLL-пекло (DLL hell): Проблеми з версіями, доступністю і збільшенням кількості DLL, особливо в Microsoft Windows;
- Пекло залежностей (Dependency hell): Проблеми з версіями потрібних продуктів, особливо в системах UNIX / GNU / Linux;

Організаційні анти-патерни:

- Аналітичний параліч (Analysis paralysis): невиправдано великі витрати на аналіз і проектування. Часто призводить до закриття проекту до початку його реалізації;
- Дійна корова (Cash cow): коли при наявності продукту, що приносить вигоду без істотних вкладень, не вкладаються кошти в розвиток і розробку нових продуктів;
- Тривале старіння (Continuous obsolescence): виділення непропорційно великих зусиль на портування системи в нові оточення;
- Скидування витрат (Cost migration): перенесення витрат на проект уразливого відділу або бізнес-партнеру;

Патерн Prototype:

Призначення патерну: Шаблон "prototype" (прототип) використовується для створення об'єктів за "шаблоном" (чи "кресленню", "ескізу") шляхом копіювання шаблонного об'єкту. Для цього визначається метод "клонувати" в об'єктах цього класу. Цей шаблон зручно використати, коли заздалегідь відомо як виглядатиме кінцевий об'єкт (мінімізується кількість змін до об'єкту шляхом створення шаблону), а також для видалення необхідності створення об'єкту - створення відбувається за рахунок клонування, і зухвалій програмі абсолютно немає необхідності знати, як створювати об'єкт. Також, це дозволяє маніпулювати об'єктами під час виконання програми шляхом налаштування відповідних шаблонів; значно зменшується ієрархія спадкоємства (оскільки в іншому випадку це були б не шаблони, а вкладені класи, що наслідують).

```
public interface CloneableImage { no usages 1 implementation
    Image clone(); 1 implementation
}
```

Рис.1 Інтерфейс CloneableImage

```
public class Image implements CloneableImage {
```

```
@Override
public Image clone() {
    try {
        Image clonedImage = new Image();
        clonedImage.setId(this.id);
        clonedImage.setImageData(this.imageData != null ? this.imageData.clone() : null);
        clonedImage.setFilename(this.filename);
        clonedImage.setContentType(this.contentType);
        clonedImage.setUser(this.user);
        return clonedImage;
    } catch (Exception e) {
        throw new RuntimeException("Failed to clone Image object.", e);
    }
}
```

Рис.2 Реалізація інтерфейсу CloneableImage

```

public Long applyEffect(Long imageId) throws IOException { 3 usages
    Image image = imageRepository.findById(imageId).orElseThrow(() -> new IOException("Image not found"));

    // Конуємо зображення перед застосуванням ефекту
    Image clonedImage = image.clone();

    BufferedImage bufferedImage = ImageIO.read(new ByteArrayInputStream(clonedImage.getImageData()));

    // Застосовуємо ефект до копії зображення
    byte[] effectData = currentState.applyEffect(bufferedImage).getImageData();

    // Оновлюємо дані зображення з новими ефектами
    clonedImage.setImageData(effectData);

    return clonedImage.getId();
}

```

Рис.3 Застосування клонування зображення

Патерн Prototype був використаний для створення копій об'єктів зображень із застосуванням різних ефектів без змін оригінальних об'єктів. Замість того щоб змінювати оригінальне зображення безпосередньо, ми використовуємо методи для копіювання (клонування) поточного зображення, на якому потім застосовуються ефекти.

Клас, що реалізує цей патерн, створює новий об'єкт із тим самим станом, що й оригінал, але вже з застосованими змінами (наприклад, поворотом, зміною розміру чи обрізанням).

Таким чином, Prototype дозволяє створювати нові версії зображення без зміни оригіналу, що є важливим для підтримки гнучкості та забезпечення стабільності в обробці даних.

Висновок: У цій роботі було розглянуто ключові аспекти проектування програмного забезпечення, зокрема популярні шаблони «Adapter», «Builder», «Command», «Chain of Responsibility» та «Prototype», а також антипатерни — приклади неправильних рішень, яких слід уникати. Окрема увага приділена реалізації шаблону «Prototype», який дозволяє створювати нові об'єкти шляхом копіювання існуючих, що підвищує ефективність і спрощує процес розробки. Цей шаблон було впроваджено для оптимізації роботи з зображеннями, для яких застосовуються ефекти. Завдяки застосуванню prototype, було покращено структуру і гнучкість коду.